

UNIT-3: Python interaction with SQLite

3.1 Module: Concepts of module and Using modules in python.

3.1.1 Concept of module

- Module is a file with '.py' extension that contain set of functions which can be imported in main program or application.
- Module is a file containing multiple definitions and statements of python.
- The created module can be imported in any of the python program.
- The module is executed first when it is imported in program.
- Each module has its own variables which can be access by other program globally as "module_name.var_name".

3.1.2 Create module

- To create module, simply write code into file with '.py' extension.
- Example: (num.py)

```
def addnum(n1,n2):
    return (n1+n2)
def subnum(n1,n2):
    return (n1-n2)
def mulnum(n1,n2):
    return(n1*n2)
```

3.1.3 Import module:

- The created modules can be used in another program in python.
- Syntax:
import module1, module2,...
- Example: (addition.py)

```
import num
n1=int(input("Enter first no:"))
n2=int(input("Enter second no:"))
a=num.addnum(n1,n2)
print (a)
a=num.subnum(n1,n2)
print(a)
a=num.mulnum(n1,n2)
print(a)
```

- Assigning alias name to module in program
- Example: (addition.py)

```
import num as n
n1=int(input("Enter first no:"))
```

```
n2=int(input("Enter second no:"))
a=n.addnum(n1,n2)
print (a)
a=n.subnum(n1,n2)
print(a)
a=n.mulnum(n1,n2)
print(a)
```

3.1.4 From-import module:

- From statement allows user to import specific attribute of module in place of whole module.
- Syntax:
from module import name1,name2,...

- Example: There is a need to import only ADDNUM function of NUM module

```
from num import addnum
n1=int(input("Enter first no:"))
n2=int(input("Enter second no:"))
a=addnum(n1,n2)
print (a)
```

(No need to specify module name before function name.)

- All attributes of module can also be imported by specifying following statement:
from num import *

3.1.5 Built in modules

- Python allows multiple built in modules which can be used as per our requirement in same format as discussed in earlier points.
- Example:

```
from math import sqrt
n1=int(input("Enter first no:"))
a=sqrt(n1)
print (a)
```

3.1.6 PYTHONPATH

- When any module is imported into python file, python interpreter finds module in local directory (folder). If it is not found there then it finds the same into all directories which are set through PYTHONPATH.
- By default, all paths from where python finds modules are stored in sys.path
- Syntax: (Check through command prompt)
Import sys
Sys.path

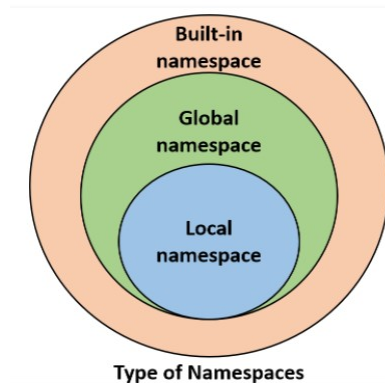
- PYTHONPATH is the environment variable which includes list of directories. (Same like PATH)
- It is used to set the path for user defined modules.
- Syntax:
Set PYTHONPATH = actual_path;
- If PYTHONPATH is set then python interpreter includes those paths into sys.path.
- Using sys.path.insert() -new path can be inserted in sys.path.

3.1.7 Namespace

- Namespace is the space where all unique names are available like function name, program name, variable name, module name etc.
- Because of this namespace, programmer can use the same variable name used in different imported modules.
- If one single program is there then all the names can be unique. But when we import modules then it may be possible that the name in two different modules may be same. Using this namespace, programmer can differentiate these two names using their module name.
- Example:

P1.py	P2.py
def print_prog() Print("Program 1")	def print_prog() Print("Program 2")
P3.py <pre> Import p1 Import p2 p1.print_prog() p2.print_prog() </pre> Output: Program1 Program2	

- Types of Namespace:

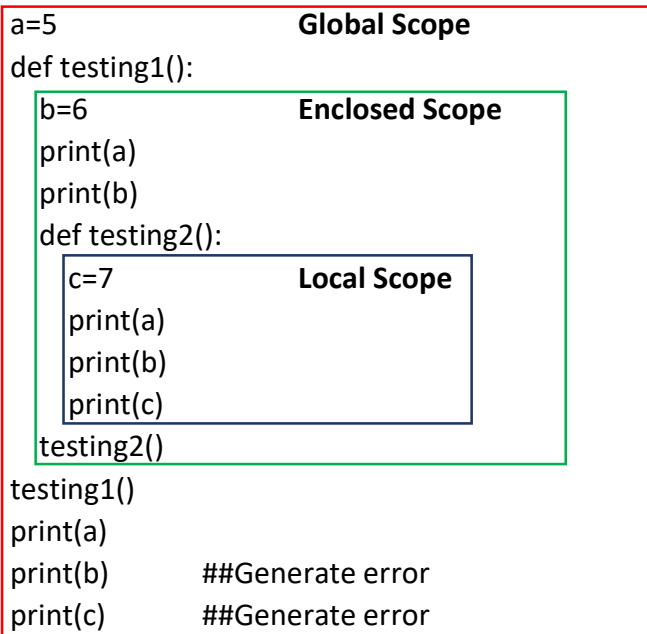


- Built-in Namespace: All identifiers which are in built use by the system.
- Global Namespace: When user creates module then the global namespace is created.
- Local Namespace: When internally any identifier is created then it is a part of local namespace.

- When the same identifier names are found in local and global namespace then using respective namespace, the name is used.

3.1.8 Scope

- Scope of object or variable name means coding block within which the object or variable can be accessible.
- Different scopes:
 - Local Scope:
 - It includes the names which are defined inside the functions.
 - This variable's scope is only within the given function.
 - Outside the functions, there is no any scope of this type of functions.
 - Global Scope:
 - It includes the names which are defined at the top of the module.
 - This variables' scope is available throughout the execution of whole module.
 - If in module, other functions are also available then these variables can also be accessible within functions.
 - GLOBAL Keyword is used to modify the respective value.
 - Enclosing Scope (Nonlocal):
 - It includes names which are defined inside any and all enclosed functions.
 - Means names which are used within nested loop.
 - NONLOCAL keyword is used to modify the respective value.



Output: 5,6,5,6,7,5,error message, error message

```

a=5
def testing1():
    b=6
    global a
    a=a+10
    print(a)
    print(b)
    def testing2():
        c=7
        nonlocal b
        global a
        b=b+10
        a=a+10
        print(a)
        print(b)
        print(c)

    testing2()

testing1()
print(a)

```

Output:

```

15
6
25
16
7
25

```

- When same variable name is available in different functions of single program, then it searches first in Local Scope, if not available then in Enclosed Scope and if not available then in Global Scope.

<pre> a=5 def testing1(): a=6 def testing2(): a=7 print(a) testing2() testing1() </pre>	<pre> a=5 def testing1(): a=6 def testing2(): print(a) testing2() testing1() </pre>	<pre> a=5 def testing1(): def testing2(): print(a) testing2() testing1() </pre>
Output: 7 Local Scope value	Output: 6 Enclosed Scope value	Output: 5 Global Scope value

3.1.9 Packages in python

- Package is a directory which is used to keep multiple modules (python files) for better usage.
- For example:
 - o Modules: p1.py, p2.py, p3.py (Different python files)
 - o Package: practical (Folder where above mentioned files are stored)
- Means maintain "practical" directory where p1, p2 and p3 modules are stored.

3.1.9.1 How to access:

- Syntax: package_name.module_name.function_name
- Example:

```
import practical.p1, practical.p2
```

practical.p1.add()	(add() is a function in p1 module)
practical.p2.sp()	(sp() is a function in p2 module)

- Syntax: from package_name import module1, module2,...
- Example:

```
from practical import p1, p2
p1.add()
p2.sp()
```

3.1.9.2 Package initialization:

- To initialize the package one file is required in same directory named “__init__.py”.
- This file is executed when the package or module of package is imported in python file.
- Directly creates __init__.py file into package related directory.
- Example:
Package: practical (It includes 4 files)
 - o File: __init__.py
 - o Modules: p1.py, p2.py, p3.py

3.2 importing sqlite3 module

- Sqlite3 module can be imported into Python.
- The SQLite database can be accessed from Python language.
- Database as well as tables can be created from python program.
- Insert, Update, Delete, Select all these operations with this database can also be allowed through python program.
- To import sqlite3 module, write following:
import sqlite3
- It imports the sqlite3 module into python which allows all variables as well as functions of sqlite3 module to be accessed from python program.

3.2.1 connect() method

- To establish connection with sqlite database, connect() method is used.
- If database is not available then it creates new database with given name.
- Using connect() method, connection object is created
- Syntax:
Var_name=sqlite3.connect('database_name')
- Example:

```
import sqlite3
conn=sqlite3.connect('emp.db')
print("Successfully connected")
print(conn)
```

Output:
Successfully connected

<sqlite3.Connection object at 0x0406CA20>

- Python finds the emp.db database, then it opens the existing database and if it is not found then it creates new database.

3.2.2 cursor() method (Not in syllabus)

- Python allows sqlite3.cursor class.
- This class allows to perform different operations within SQLite database.
- It allows multiple methods to execute SQLite statements.
- Cursor object is created using cursor() method.
- Syntax:
`Connection_name.cursor()`
- Example:
`conn.cursor()`
- It allows following methods:
 - o `execute()`
 - o `fetchone()`
 - o `fetchall()`
- If user doesn't create cursor object, the python automatically creates cursor implicitly.
 - o `A=curs.execute()` #explicitly cursor is created
 - o `B=conn.execute()` #automatically cursor is created

3.2.3 execute() method

- To execute the SQL statements means operations like INSERT, UPDATE, DELETE etc., `execute()` method of cursor object is used.
- Syntax:
 - o Explicitly cursor is created:
`Var_name=cursor_name.execute("SQL_statement")`
 - o Automatically cursor is created:
`Var_name=connection_name.execute("SQL_statement")`

3.2.4 fetchone() method

- `fetchone()` method extract one record from current position.
- First time it extracts first record. If again it is executed then it extracts second record.
- If not a single record exists or after the last record, it is executed then it returns "None".
- It generates one dimensional array.
- Example:

```
import sqlite3
conn=sqlite3.connect('emp.db')
a=conn.execute("select * from tblemp") #automatically cursor object is created.
b=a.fetchone()
print(b)
```

```

print("Emp id:",b[0])
print("Emp. name:",b[1])
print("Emp. gender:",b[2])
print("Emp. join date:", b[3])
print("Emp. salary:", b[4])
print(a.fetchone())

```

Output:

```

(3, 'Geeta', 'F', '2010-08-20', 25000)
Emp id: 3
Emp. name: Geeta
Emp. gender: F
Emp. join date: 2010-08-20
Emp. salary: 25000
(4, 'Jatin', 'M', '2011-04-12', 25000)

```

3.2.5 fetchall() method

- fetchall() method is used to extract all records based on given query.
- It generates two dimensional array. 9638901138
- Example

```

import sqlite3
conn=sqlite3.connect('emp.db')
a=conn.execute("select * from tblemp") #automatically cursor object is created.

b=a.fetchall()

print(b)

print("Emp id:",b[0][0])
print("Emp. name:",b[0][1])
print("Emp. gender:",b[0][2])
print("Emp. join date:", b[0][3])
print("Emp. salary:", b[0][4],"\n")

print("Emp id:",b[1][0])
print("Emp. name:",b[1][1])
print("Emp. gender:",b[1][2])
print("Emp. join date:", b[1][3])
print("Emp. salary:", b[1][4],"\n")

```

Output:

```

[(3, 'Geeta', 'F', '2010-08-20', 25000), (4, 'Jatin', 'M', '2011-04-12', 25000),
(5, 'Prashant', 'M', '2009-07-10', 25000)]
Emp id: 3

```


Emp. name: Geeta
 Emp. gender: F
 Emp. join date: 2010-08-20
 Emp. salary: 25000

Emp id: 4
 Emp. name: Jatin
 Emp. gender: M
 Emp. join date: 2011-04-12
 Emp. salary: 25000

3.2.6 `commit()` method

- When any data is modified within database (using Insert, Update, Delete operations) then there is a need to save the data permanently within table.
- Compulsory, programmer has to write commit to save current changes permanently into database.
- Python allows `commit()` method under connection object.
- Syntax:
`Conn_name.commit()`
- Example:
`conn.commit()`
- If programmer tries to access the changes without committing data, then he is not able to access it.

3.2.7 Creating table within database:

- CREATE statement is used to create table within database.
- Python allows programmer to create table using SQL statement using `execute()` method.
- Example:

```
import sqlite3

conn=sqlite3.connect('emp.db')

conn.execute("create table tblemp(eno int primary key, ename text, gender text,
jdate text, salary integer)")

print("Table successfully created")
```

Output:

Table successfully created

3.2.8 Inserting data into database:

- To insert the records into existing table, INSERT statement is used.

- The same insert statement can be written within python program using execute() method.
- Example:

```
import sqlite3
conn=sqlite3.connect('emp.db')
#c=conn.execute("create table tblemp(eno int primary key, ename text, gender text,
jdate text, salary integer)")
conn.execute("insert into tblemp values(1,'Ronit','M','2001-12-21',25000)")
conn.execute("insert into tblemp values(2,'Roghav','M','2001-12-21',25000)")
conn.execute("insert into tblemp values(3,'Geeta','F','2010-08-20',25000)")
conn.execute("insert into tblemp values(4,'Jatin','M','2011-04-12',25000)")
conn.execute("insert into tblemp values(5,'Prashant','M','2009-07-10',25000)")
print("Records successfully inserted")
conn.commit()
conn.close()
```

Output:

Records successfully inserted

3.2.9 For loop in python (Not in syllabus)

- For loop is used to execute block of statements multiple times.
- Syntax:


```
for var in range:
    statements
```
- Example: "Print Hello 5 times"

```
for i in range(0,5):
    print("Hello")
```

3.2.10 Reading data from database:

- SELECT statement is used to fetch or read or select data from SQLite database.
- Python allows execute() method, using which the data can be fetched from existing table.
- Example

```
import sqlite3
conn=sqlite3.connect('emp.db')
r=conn.execute("select * from tblemp")
print(r)
for i in r:
    print("Employee id:",i[0],"Employee name:",i[1],"Gender:",i[2],"Salary:",i[4])
print("records successfully displayed")
conn.close()
```

Output:

<sqlite3.Cursor object at 0x0399A7A0>

Employee id: 2 Employee name: Roghav Gender: M Salary: 25000

Employee id: 3 Employee name: Geeta Gender: F Salary: 25000

Employee id: 4 Employee name: Jatin Gender: M Salary: 25000

Employee id: 5 Employee name: Prashant Gender: M Salary: 25000

records successfully displayed

- Python automatically creates a CURSER object – See output of Print(r)

3.2.11 Updating data into database:

- UPDATE statement is used to update data of database.
- Python allows to update existing data of SQLite database using execute() method.
- Example

```
import sqlite3
conn=sqlite3.connect('emp.db')
conn.execute("update tblemp set salary=40000 where eno=2")
print("Records successfully updated")
conn.commit()
```

Output:

Record successfully updated.

3.2.12 Deleting data from database:

- DELETE statement is used to delete the records from existing tables.
- Python allows to use the same statement using execute() method.
- Example

```
import sqlite3
conn=sqlite3.connect('emp.db')
conn.execute("delete from tblemp where eno=2")
print("Record successfully deleted")
conn.commit()
```

Output:

Record successfully deleted.

3.3 Inserting user data into sqlite database

- Python allows to take input from user. Same data can be inserted into SQLite existing tables.
- Example:

```
#tblemp(eno,name,gender,jdate,salary)

import sqlite3 as sq
c=sq.connect('c:/SQLite/emp.db')
e1=int(input("Enter employee no:"))
e2=input("Enter employee name:")
e3=input("Enter employee's gender:")
e4=input("Enter Joining date:")
e5=int(input("Enter salary:"))

#insert the record as per user's input
c.execute(f"insert into tblemp values {e1,e2,e3,e4,e5}")

c.commit()
```

3.4 Fetch data as per user's requirement

- Python allows to display data as per user's requirement.

3.4.1 As per integer values:

- Example:

```
import sqlite3 as sq
c=sq.connect('c:/SQLite/emp.db')
num=int(input("Enter Employee no:"))
a=c.execute(f"select * from tblemp where eno={num}")
for i in a:
    print(i)
```

3.4.2 As per text values:

- Example:

```
import sqlite3 as sq
c=sq.connect('c:/SQLite/emp.db')
nm=input("Enter employee gender:")
a=c.execute(f"select * from tblemp where gender='{nm}' ")
for i in a:
    print(i)
```